

Phase 2 Backend Implementation Guide

1. What This Document Is For

This is the ****detailed Phase 2 backend implementation guide**** for the Aivan HRMS Portal.

It is written for a developer who may still be building confidence with backend work and needs a practical roadmap.

This guide explains:

- what Phase 2 backend should add
- what existing backend code can be reused
- which modules should be completed first
- what database tables should be added or extended
- which APIs frontend will need
- how to implement the workflows safely

2. Current Backend Baseline

The backend already contains these important pieces:

- FastAPI app
- auth routes
- employee routes
- HR routes
- attendance routes
- dashboard routes
- profile routes
- notification routes
- login activity routes
- partial time off implementation
- migrations
- services and repository-style structure

This means Phase 2 backend should be approached as:

- complete existing partial modules
- normalize contracts
- extend schema cleanly
- improve workflow integrity

It should ****not**** be treated as a brand-new backend project.

3. Phase 2 Backend Goal

Phase 2 backend should make the product operationally stronger by introducing:

- request and approval workflows
- admin-managed master data
- audit-friendly state changes
- reporting endpoints
- cleaner ownership of filtering, pagination, and aggregates

4. Phase 2 Backend Scope

Must Build

1. complete time off module
2. approval center APIs
3. attendance regularization workflow
4. master data CRUD APIs
5. report and export APIs
6. stronger notification integration
7. refined login activity and security flows

Must Improve

1. server-side filtering
2. server-side pagination
3. response consistency
4. role-based authorization clarity
5. migration discipline

Not Required In Phase 2

- payroll engine
- payslip generation
- reimbursement accounting
- appraisal engine
- recruitment workflow
- multi-company architecture

5. First Important Rule

Do not build Phase 2 as one giant module.

Build by workflow.

Recommended order:

1. cleanup and shared infrastructure
2. time off
3. approval center
4. attendance regularization
5. master data
6. reports
7. security refinements

This is important because later modules depend on earlier ones.

6. Recommended Backend Architecture For Phase 2

Keep the existing modular monolith structure.

Recommended layer pattern:

```
'''text
Route -> Schema -> Service -> Repository/Model -> Database
'''
```

Route Layer

Only HTTP details should live here:

- path params

- query params
- auth dependencies
- response models

Service Layer

Business rules should live here:

- request validation
- approval rules
- balance calculations
- status transitions
- notification triggering

Repository Layer

DB query patterns should live here when queries become complex:

- filtered lists
- summary aggregates
- export queries

7. Backend Folder Expansion

Suggested expansion of the current backend:

```
'''text
backend/app/
??? api/v1/
?   ??? approval_routes.py
?   ??? master_data_routes.py
?   ??? regularization_routes.py
?   ??? report_routes.py
?   ??? timeoff_routes.py
??? models/
?   ??? attendance_regularization.py
?   ??? approval_task.py
?   ??? department.py
?   ??? designation.py
?   ??? shift.py
?   ??? work_location.py
?   ??? leave_type.py
?   ??? holiday_calendar.py
??? schemas/
?   ??? approval.py
?   ??? regularization.py
?   ??? master_data.py
?   ??? report.py
??? services/
?   ??? approval_service.py
?   ??? regularization_service.py
?   ??? master_data_service.py
?   ??? report_service.py
??? repositories/
    ??? approval_repository.py
    ??? regularization_repository.py
    ??? master_data_repository.py
    ??? report_repository.py
'''
```

8. Core Workflow Design

Phase 2 backend revolves around three important workflow types:

1. request workflows
2. approval workflows
3. audit workflows

Request Workflow

Example:

- employee submits time off
- employee submits attendance correction

Approval Workflow

Example:

- HR reviews request
- HR approves or rejects
- admin may review escalated items

Audit Workflow

Every important action should leave a trace:

- who submitted
- who approved or rejected
- when it happened
- what changed
- comment or reason if relevant

9. Phase 2 Master Data Design

Phase 1 forms already need stable dropdown values.

Phase 2 should move these into database-managed master tables.

Required Master Tables

1. 'departments'
2. 'designations'
3. 'shifts'
4. 'work_locations'
5. 'leave_types'
6. 'holidays'

Suggested Common Columns

Each master table should have:

- 'id'
- 'name'
- 'code'
- 'description'
- 'is_active'
- 'created_at'
- 'updated_at'
- 'created_by'

- 'updated_by'

This keeps behavior consistent across admin-managed reference data.

10. Exact Database Schema Additions

10.1 'leave_types'

Purpose:

- stores allowed leave categories

Columns:

- 'id' bigint primary key
- 'name' varchar(100) not null unique
- 'code' varchar(30) not null unique
- 'unit_type' varchar(20) not null
- 'default_balance_hours' numeric(10,2) not null default 0
- 'requires_approval' boolean not null default true
- 'is_active' boolean not null default true
- 'created_at' timestamptz not null default now()
- 'updated_at' timestamptz not null default now()

Examples:

- Casual Leave
- Sick Leave
- Work From Home
- Comp Off

10.2 'holidays'

Purpose:

- stores holiday calendar

Columns:

- 'id' bigint primary key
- 'holiday_date' date not null unique
- 'name' varchar(120) not null
- 'description' text null
- 'is_optional' boolean not null default false
- 'is_active' boolean not null default true
- 'created_at' timestamptz not null default now()
- 'updated_at' timestamptz not null default now()

10.3 'attendance_regularization_requests'

Purpose:

- employee asks HR to correct missed or wrong attendance

Columns:

- 'id' bigint primary key
- 'employee_id' bigint not null references employees(id)
- 'attendance_date' date not null
- 'requested_punch_in' time null
- 'requested_punch_out' time null

- 'reason_type' varchar(50) not null
- 'reason_text' text not null
- 'status' varchar(20) not null default 'pending'
- 'reviewed_by' bigint null references users(id)
- 'reviewed_at' timestampz null
- 'review_comment' text null
- 'created_at' timestampz not null default now()
- 'updated_at' timestampz not null default now()

Important unique rule:

- one active pending request per employee per attendance date

10.4 'approval_tasks'

Purpose:

- creates one central queue for requests needing approval

Columns:

- 'id' bigint primary key
- 'request_type' varchar(30) not null
- 'request_id' bigint not null
- 'employee_id' bigint not null references employees(id)
- 'assigned_role' varchar(20) not null default 'hr'
- 'status' varchar(20) not null default 'pending'
- 'priority' varchar(20) not null default 'normal'
- 'submitted_by' bigint not null references users(id)
- 'reviewed_by' bigint null references users(id)
- 'reviewed_at' timestampz null
- 'decision_comment' text null
- 'created_at' timestampz not null default now()
- 'updated_at' timestampz not null default now()

Important note:

This table lets frontend show one single approval center instead of reading separate tables directly.

10.5 Extend 'timeoff_requests'

Current table already exists.

Add or confirm these fields:

- 'leave_type_id' bigint null references leave_types(id)
- 'status' varchar(20) not null default 'pending'
- 'reviewed_by' bigint null references users(id)
- 'reviewed_at' timestampz null
- 'review_comment' text null
- 'requested_hours' numeric(10,2) not null default 0
- 'approved_hours' numeric(10,2) null
- 'is_cancelled' boolean not null default false

10.6 Extend 'employees'

Add or confirm:

- 'department_id'
- 'designation_id'
- 'shift_id'
- 'work_location_id'

- 'reporting_manager_id' optional
- 'employment_status'
- 'join_date'
- 'timeoff_balance_hours'

10.7 Exact SQLAlchemy And Pydantic Type Direction

This section is important for a fresher developer.

It clearly says which backend type to use.

SQLAlchemy Type Mapping

Use these standard mappings:

- 'id': 'Mapped[int]' with 'BigInteger' or 'Integer'
- text name fields: 'Mapped[str]' with 'String(length)'
- long notes: 'Mapped[str | None]' with 'Text'
- flags: 'Mapped[bool]' with 'Boolean'
- decimal hours: 'Mapped[Decimal]' or 'Mapped[float]' with 'Numeric(10, 2)'
- timestamps: 'Mapped[datetime]' with 'DateTime(timezone=True)'
- pure dates: 'Mapped[date]' with 'Date'
- pure time values: 'Mapped[time | None]' with 'Time'

Pydantic Field Type Mapping

Use these field types in request and response schemas:

- 'id': 'int'
- 'name': 'str'
- 'description': 'str | None'
- 'is_active': 'bool'
- 'date': 'date'
- 'timestamp': 'datetime'
- 'requested_hours': 'float'
- 'approved_hours': 'float | None'
- arrays: 'list[SomeSchema]'
- string maps: 'dict[str, list[str]]'

Enum Values To Standardize

Do not allow free text everywhere.

Use fixed values like:

- 'request_type': ''timeoff' | 'regularization''
- 'status': ''pending' | 'approved' | 'rejected' | 'cancelled''
- 'duration_type': ''full_day' | 'half_day' | 'hourly''
- 'assigned_role': ''hr' | 'admin''
- 'employment_status': ''active' | 'inactive' | 'notice_period' | 'relieved''

10.8 Suggested Pydantic Schemas

These should be written as dedicated schema classes.

'schemas/timeoff.py'

'''py

class TimeOffCreateRequest(BaseModel):

```

    leaveTypeId: int
    dateFrom: date
    dateTo: date
    durationType: Literal['full_day', 'half_day', 'hourly']
    startTime: time | None = None
    endTime: time | None = None
    reason: str

```

```

class TimeOffDecisionRequest(BaseModel):
    decision: Literal['approved', 'rejected']
    comment: str
    approvedHours: float | None = None

```

```

class TimeOffRequestResponse(BaseModel):
    id: int
    employeeId: int
    leaveTypeId: int
    leaveTypeName: str
    dateFrom: date
    dateTo: date
    durationType: str
    requestedHours: float
    approvedHours: float | None
    reason: str
    status: str
    reviewComment: str | None
    createdAt: datetime
    reviewedAt: datetime | None
'''

```

'schemas/regularization.py'

```

'''py
class RegularizationCreateRequest(BaseModel):
    attendanceDate: date
    requestedPunchIn: time | None = None
    requestedPunchOut: time | None = None
    reasonType: Literal[
        'missed_punch',
        'late_sync',
        'forgot_punch_out',
        'forgot_punch_in',
        'system_issue',
        'other'
    ]
    reasonText: str

```

```

class RegularizationDecisionRequest(BaseModel):
    decision: Literal['approved', 'rejected']
    comment: str
'''

```

'schemas/master_data.py'

```

'''py
class DepartmentCreateRequest(BaseModel):
    name: str
    code: str
    description: str | None = None
    isActive: bool = True

```

```

class DepartmentResponse(BaseModel):
    id: int

```



```

    name: str
    code: str
    description: str | None
    isActive: bool
'''

```

10.9 Common API Response Contract

Backend should return a predictable structure so frontend stays simple.

Success Response

```

'''json
{
  "success": true,
  "message": "Record created successfully.",
  "data": {}
}
'''

```

Validation Error Response

```

'''json
{
  "success": false,
  "message": "Validation failed.",
  "errors": {
    "leaveTypeId": ["This field is required."],
    "dateFrom": ["Invalid date."]
  }
}
'''

```

Paginated List Response

```

'''json
{
  "items": [],
  "page": 1,
  "pageSize": 20,
  "totalItems": 0,
  "totalPages": 0
}
'''

```

11. API Modules To Build

11.1 Time Off APIs

'GET /api/v1/timeoff/bootstrap'

Purpose:

- load request form data

Response:

```

'''json
{

```

```
"leaveTypes": [
  {
    "id": 1,
    "name": "Casual Leave",
    "code": "CL",
    "unitType": "full_day"
  }
],
"balance": {
  "totalHours": 80,
  "usedHours": 12,
  "remainingHours": 68
},
"holidays": [
  {
    "date": "2026-08-15",
    "name": "Independence Day"
  }
]
},
,,,
```

'POST /api/v1/timeoff/requests'

Request:

```
'''json
{
  "leaveTypeId": 1,
  "dateFrom": "2026-06-15",
  "dateTo": "2026-06-15",
  "durationType": "hourly",
  "startTime": "14:00",
  "endTime": "16:00",
  "reason": "Doctor appointment"
}
,,,
```

Response:

```
'''json
{
  "id": 101,
  "status": "pending",
  "message": "Time off request submitted successfully."
}
,,,
```

'GET /api/v1/timeoff/requests/my'

Purpose:

- employee history

'GET /api/v1/timeoff/requests'

Role:

- hr, admin

Query params:

- 'status'

- 'employeeId'
- 'dateFrom'
- 'dateTo'
- 'page'
- 'pageSize'

Query param types:

- 'status': 'string'
- 'employeeId': 'number'
- 'dateFrom': 'YYYY-MM-DD'
- 'dateTo': 'YYYY-MM-DD'
- 'page': 'number'
- 'pageSize': 'number'

'POST /api/v1/timeoff/requests/{requestId}/decision'

Request:

```
'''json
{
  "decision": "approved",
  "comment": "Approved",
  "approvedHours": 2
}
'''
```

Response:

```
'''json
{
  "requestId": 101,
  "status": "approved",
  "approvalTaskStatus": "resolved"
}
'''
```

11.2 Approval Center APIs

'GET /api/v1/approvals/pending'

Purpose:

- single queue for HR/admin

Response:

```
'''json
{
  "items": [
    {
      "id": 501,
      "requestType": "timeoff",
      "requestId": 101,
      "employeeId": 8,
      "employeeName": "Kaushal Raj",
      "status": "pending",
      "submittedAt": "2026-06-08T10:15:00Z",
      "priority": "normal"
    }
  ],
}
```

```
"counts": {
  "timeoff": 4,
  "regularization": 2,
  "total": 6
}
},
'''
```

'GET /api/v1/approvals/history'

Purpose:

- processed approval items

Query params:

- 'requestType: string | optional'
- 'employeeId: number | optional'
- 'dateFrom: string | optional'
- 'dateTo: string | optional'
- 'page: number'
- 'pageSize: number'

'POST /api/v1/approvals/{approvalTaskId}/decision'

Purpose:

- generic approval endpoint if frontend wants a single action route

Recommended note:

Keep both patterns possible:

- module-specific endpoints
- generic approval endpoint

Start with module-specific if easier.

11.3 Attendance Regularization APIs

'POST /api/v1/regularizations'

Request:

```
'''json
{
  "attendanceDate": "2026-06-04",
  "requestedPunchIn": "09:12",
  "requestedPunchOut": "18:22",
  "reasonType": "missed_punch",
  "reasonText": "Mobile battery was dead during punch-out."
}
'''
```

Response:

```
'''json
{
  "id": 301,
  "status": "pending",
  "message": "Regularization request submitted successfully."
}
```

```
}  
'''
```

'GET /api/v1/regularizations/my'

Purpose:

- employee own request history

'GET /api/v1/regularizations'

Role:

- hr, admin

Purpose:

- HR review queue

Query params:

- 'status'
- 'employeeId'
- 'attendanceDate'
- 'page'
- 'pageSize'

'POST /api/v1/regularizations/{id}/decision'

Request:

```
'''json  
{  
  "decision": "approved",  
  "comment": "Approved after review."  
}  
'''
```

Backend rule:

If approved, update the attendance record and rebuild summary values for that date.

11.4 Master Data APIs

Each master data module should support:

- list
- create
- update
- deactivate
- bootstrap read

Example Endpoints

- 'GET /api/v1/master-data/departments'
- 'POST /api/v1/master-data/departments'
- 'PUT /api/v1/master-data/departments/{id}'
- 'PATCH /api/v1/master-data/departments/{id}/status'

Do the same for:

- designations
- shifts
- work locations
- leave types
- holidays

'GET /api/v1/master-data/bootstrap'

Purpose:

- one-call dropdown source for forms

Response:

```
'''json
{
  "departments": [],
  "designations": [],
  "shifts": [],
  "workLocations": [],
  "leaveTypes": []
}
'''
```

Each key type:

- 'departments': 'DepartmentResponse[]'
- 'designations': 'DesignationResponse[]'
- 'shifts': 'ShiftResponse[]'
- 'workLocations': 'WorkLocationResponse[]'
- 'leaveTypes': 'LeaveTypeResponse[]'

11.5 Report APIs

'GET /api/v1/reports/attendance'

Query params:

- 'employeeId'
- 'departmentId'
- 'dateFrom'
- 'dateTo'
- 'status'
- 'page'
- 'pageSize'

'GET /api/v1/reports/attendance/export'

Purpose:

- CSV download

'GET /api/v1/reports/timeoff'

Purpose:

- leave usage report

'GET /api/v1/reports/exceptions'

Purpose:

- late arrivals
- missing punches
- unresolved regularizations

12. Business Rules To Enforce

12.1 Time Off Rules

- cannot request leave for inactive employee
- cannot approve beyond available balance if balance-controlled leave type
- cannot create overlapping pending full-day leave for same date
- hourly leave must stay inside shift window rules if policy requires
- only HR/admin can approve or reject

12.2 Regularization Rules

- employee can only request for allowed past date window
- duplicate pending request should be blocked
- approved correction must update final attendance summary
- rejected request must remain visible in history

12.3 Master Data Rules

- inactive master records should not appear in new form dropdowns
- old employee records should still remain readable even if master record later becomes inactive

12.4 Notification Rules

Create notifications for:

- new time off request submitted
- time off request approved/rejected
- new regularization request submitted
- regularization approved/rejected
- suspicious login or login activity event if product keeps that behavior

12.5 Validation Rules The Backend Must Enforce

Never trust frontend validation alone.

Time Off Validation

- 'leaveTypeId' must exist
- 'dateFrom' cannot be after 'dateTo'
- 'reason' cannot be blank
- if 'durationType = hourly', then 'startTime' and 'endTime' are required
- 'approvedHours' cannot be greater than 'requestedHours'

Regularization Validation

- at least one of 'requestedPunchIn' or 'requestedPunchOut' should be present
- request date should belong to employee attendance-access window
- employee must exist and be active
- duplicate pending request for same date should fail

Master Data Validation

- 'code' should be unique
- 'name' should not be blank
- inactive master data should not be hard-deleted if already mapped to employees

12.6 Service Function Checklist

This is how fresher backend developer should think.

'timeoff_service.py'

Required functions:

- 'get_timeoff_bootstrap(db, employee_id) -> dict'
- 'create_timeoff_request(db, employee_id, payload) -> TimeOffRequest'
- 'list_my_timeoff_requests(db, employee_id, filters) -> paginated result'
- 'list_timeoff_requests_for_hr(db, filters) -> paginated result'
- 'decide_timeoff_request(db, request_id, reviewer_id, payload) -> TimeOffRequest'

'regularization_service.py'

Required functions:

- 'create_regularization_request(db, employee_id, payload)'
- 'list_my_regularizations(db, employee_id, filters)'
- 'list_regularizations_for_hr(db, filters)'
- 'decide_regularization_request(db, request_id, reviewer_id, payload)'
- 'apply_regularization_to_attendance(db, request_row)'

'master_data_service.py'

Required functions:

- 'list_departments(db, filters)'
- 'create_department(db, payload, actor_id)'
- 'update_department(db, id, payload, actor_id)'
- 'change_department_status(db, id, is_active, actor_id)'
- same pattern for all other master data tables

12.7 Repository Query Responsibility

Move these heavy tasks into repository layer:

- paginated list queries
- filtered report queries
- count aggregations
- dashboard summaries
- joins across employee, attendance, department, and request tables

This helps the fresher developer keep service files readable.

13. How Notifications Should Be Wired

Every important workflow event should call one notification helper.

Recommended approach:

1. service updates the main business record

2. service updates approval task
3. service writes audit log
4. service triggers notification
5. optional websocket push for live UI

This order helps keep system behavior consistent.

14. Audit Logging Plan

For Phase 2, auditability becomes more important.

For every approval decision, store:

- request type
- request id
- old status
- new status
- actor user id
- actor role
- comment
- timestamp

You can use:

- a generic 'approval_logs' table
- or expand existing audit structures

If the current 'approval_log' model is already present, extend and reuse it rather than duplicating the idea.

15. Pagination and Filtering Standard

All list APIs in Phase 2 should follow one standard contract.

Recommended response:

```
'''json
{
  "items": [],
  "page": 1,
  "pageSize": 20,
  "totalItems": 128,
  "totalPages": 7
}
'''
```

Use this for:

- employee lists
- HR lists
- leave requests
- regularization requests
- reports
- login activity

This is important because Phase 1 currently has too much list handling on the frontend side.

16. Step-By-Step Backend Build Order

Step 1. Cleanup Existing Contracts

Before adding new modules:

- review current auth responses
- review attendance list response shapes
- review notification route consistency
- confirm role dependency helpers

Step 2. Finish Time Off Schema and Routes

Do this first because some backend parts already exist.

Tasks:

- activate router in 'main.py'
- normalize schemas
- finish service rules
- add approval integration
- seed leave types

Step 3. Add Approval Center Layer

Tasks:

- create 'approval_tasks'
- create approval list APIs
- centralize approval status transitions

Step 4. Add Attendance Regularization

Tasks:

- create model and migration
- create employee submit API
- create HR review API
- update attendance rebuild logic

Step 5. Build Master Data CRUD

Tasks:

- departments
- designations
- shifts
- work locations
- leave types
- holidays

Step 6. Build Reports

Tasks:

- aggregated attendance queries
- exception queries
- time off summaries
- CSV export

Step 7. Refine Notifications and Security

Tasks:

- make workflow notifications consistent
- improve login activity filters
- confirm reset-password and forgot-password production behavior

17. Seed Data For Phase 2

Add seed data for:

- default leave types
- departments
- designations
- shifts
- office locations
- current year holiday list

Example leave types:

- Casual Leave
- Sick Leave
- Half Day
- Work From Home
- Comp Off

Example departments:

- Engineering
- HR
- Finance
- Operations

Example shifts:

- General Shift 9 to 6
- Morning Shift 8 to 5
- Flexible Shift

18. Testing Checklist

For each module, test:

Time Off

- employee submits request
- HR approves
- HR rejects
- balance updates correctly
- duplicate or invalid requests fail

Regularization

- employee submits request
- duplicate request blocked
- HR approves
- attendance recalculated
- HR rejects

Master Data

- create record
- update record
- deactivate record
- bootstrap excludes inactive records

Reports

- filters work
- pagination works
- export works

19. Definition Of Done For Phase 2 Backend

Phase 2 backend is done when:

- all new workflows are backed by DB tables and migrations
- list APIs support filtering and pagination
- approval actions update both business records and audit records
- frontend no longer has to simulate backend-heavy list logic
- notifications are triggered from workflow events
- time off router is fully live
- regularization flows are complete
- master data is controlled by admin APIs
- report APIs return usable data for tables and export

20. Advice For The Developer

If one fresher developer is handling both frontend and backend:

- do backend data models first
- do not start with report screens
- finish one workflow completely before opening the next

Best order:

1. time off full backend
2. time off frontend
3. approval center backend
4. approval center frontend
5. regularization backend
6. regularization frontend
7. master data backend
8. master data frontend
9. reports backend
10. reports frontend

That path is the safest and easiest to reason about.

21. Fresher Backend Start Plan

If the developer is new to backend, tell him to start exactly like this:

Day 1

- read current route files
- understand how 'auth_routes.py', 'employee_routes.py', and 'attendance_routes.py' are structured

- run the backend locally
- confirm DB connection and migrations

Day 2

- write only schemas for one module first
- for example start with 'timeoff'
- define request schema
- define response schema
- define success/error structure

Day 3

- create or update model and migration
- check table columns carefully
- do not build route first

Day 4

- build service methods
- add validations
- test with Swagger or Postman

Day 5

- build route file
- connect route to service
- test full flow

Important rule:

Always build in this order:

1. schema
2. model
3. migration
4. service
5. route
6. manual API test

This avoids confusion and reduces rework a lot.